

Carlus Electum

Software Architecture Document

Version 1.0 approved

Prepared by João

26/12/2025

Indice

1. SOFTWARE ARCHITECTURE PRINCIPLES.....	3
1.1 Modularity	3
1.2 Cohesion	3
1.3 Coupling	3
1.4 Separation of Concerns	4
2. SOLID PRINCIPLES APPLICATION.....	4
3. DESIGN PATTERNS	5
3.1 Strategy Pattern	5
3.2 Factory Pattern.....	6
3.3 Observer Pattern	7
5. TECHNOLOGY STACK JUSTIFICATION.....	8

1. SOFTWARE ARCHITECTURE PRINCIPLES

A fundação arquitetural da Carlus Electum baseia-se em princípios sólidos de engenharia de software que garantem a viabilidade do sistema a longo prazo.

1.1 Modularity

A prática de dividir o sistema em componentes independentes é central na Carlus Electum. O sistema está segmentado em módulos distintos como Gestão de Autenticação, Processamento de Pagamentos, Catálogo de Produtos e Gestão de Inventário.

Independência: Cada módulo encapsula uma funcionalidade específica, permitindo o desenvolvimento paralelo por diferentes membros da equipa.

Facilidade de Teste: Módulos isolados simplificam a criação de testes unitários com JUnit, permitindo focar na lógica interna de cada componente.

1.2 Cohesion

Buscamos uma Alta Coesão, onde cada classe e módulo na Carlus Electum possui uma única e bem definida responsabilidade.

Por exemplo, a classe `InventoryService` foca-se exclusivamente na monitorização de stock e reserva de itens, não interferindo com a lógica de renderização da interface ou detalhes de pagamento.

1.3 Coupling

O design visa o Baixo Acoplamento, minimizando as dependências entre módulos.

Através do uso de interfaces, as camadas superiores não dependem de implementações concretas das camadas inferiores, o que facilita a manutenção e substituição de componentes (ex: trocar de base de dados ou gateway de pagamento) sem afetar o resto do sistema.

1.4 Separation of Concerns

O sistema está organizado em três camadas lógicas:

1. Presentation Layer: Gere a interação com o cliente e administrador.
2. Business Logic Layer: Implementa as regras de negócio de e-commerce (preços, estados de encomenda).
3. Data Layer: Responsável pela persistência e integridade dos dados na base de dados relacional.

2.SOLID PRINCIPLES APPLICATION

A aplicação dos princípios SOLID garante que as classes da Carlus Electum sejam resilientes a mudanças.

S - Single Responsibility Principle: Cada classe tem uma única razão para mudar. A classe OrderService foca-se apenas no fluxo de estados da encomenda, enquanto NotificationService gere as comunicações por email.

O - Open-Closed Principle: O sistema está aberto para extensão, mas fechado para modificação. O padrão Strategy aqui aplicado permite adicionar novas promoções sem alterar o código do carrinho.

L - Liskov Substitution Principle: A classe Administrador pode ser usada em qualquer lugar onde se espera um Utilizador, garantindo que as permissões e autenticação funcionam sem erros.

I - Interface Segregation Principle: Criámos interfaces específicas (ex: IOrderTrackable, IPayable) para que o Operador de Logística não dependa de métodos de pagamento que não lhe competem.

D - Dependency Inversion Principle: Módulos de alto nível não dependem de módulos de baixo nível, mas sim de abstrações (interfaces).

3. DESIGN PATTERNS

3.1 Strategy Pattern

O padrão **Strategy** é fundamental para cumprir os requisitos de promoções flexíveis (FR-015, FR-016). Abaixo, apresentamos a implementação integral da lógica de negócio para a campanha "Leve 3 Pague 2", onde o sistema identifica automaticamente o item mais barato para oferecer ao cliente.

Implementação Java Completa:

```
import java.util.List;
import java.util.Collections;

public interface DiscountStrategy {
    double applyDiscount(double total);
}

/**
 * Implementação de desconto percentual fixo.
 */
public class PercentageDiscount implements DiscountStrategy {
    private double rate;

    public PercentageDiscount(double rate) { this.rate = rate; }

    @Override
    public double applyDiscount(double total) {
        return total * (1 - rate / 100);
    }
}
```

```

/**
 * Implementação da lógica "Leve 3 Pague 2".
 * A lógica desconta o item mais barato de cada conjunto de três itens.
 */
public class Buy2Get3Discount implements DiscountStrategy {
    private List<Double> itemPrices;

    // Recebe a lista de preços dos itens no carrinho
    public Buy2Get3Discount(List<Double> itemPrices) {
        this.itemPrices = itemPrices;
    }

    @Override
    public double applyDiscount(double total) {
        if (itemPrices == null || itemPrices.size() < 3) {
            return total; // Não há itens suficientes para o desconto
        }

        // Ordenar os preços para garantir que o desconto é feito nos mais baratos
        Collections.sort(itemPrices);

        int numberOfFreeItems = itemPrices.size() / 3;
        double discountAmount = 0;

        // Soma o preço dos itens que serão gratuitos (os mais baratos)
        for (int i = 0; i < numberOfFreeItems; i++) {
            discountAmount += itemPrices.get(i);
        }

        double finalPrice = total - discountAmount;
        System.out.println("Campanha Leve 3 Pague 2: Desconto de " + discountAmount + "€ aplicado.");

        return finalPrice;
    }
}

```

3.2 Factory Pattern

Fornece uma interface para criar objetos sem especificar as suas classes concretas. Na Carlsberg, é usado para criar o método de pagamento correto (PayPal, Cartão) conforme a escolha do cliente.

```
public class PaymentFactory {  
    public static PaymentProcessor createPayment(String type) {  
        if (type.equalsIgnoreCase("PAYPAL")) return new PayPalProcessor();  
        if (type.equalsIgnoreCase("CARD")) return new CreditCardProcessor();  
        else throw new IllegalArgumentException("Método de pagamento desconhecido.");  
    }  
}
```

3.3 Observer Pattern

Garante que o estado da encomenda seja refletido automaticamente em múltiplos sistemas (Email, Auditoria, Logística).

5. TECHNOLOGY STACK JUSTIFICATION

A escolha das tecnologias para a Carlus Electum foi guiada pelos requisitos de qualidade e segurança definidos no enunciado.

Linguagem Java: A escolha do Java justifica-se pela sua robustez na implementação de padrões de desenho e pela tipagem forte, que previne erros comuns em tempo de execução, vitais para a segurança financeira (NFR-005).

JUnit 5: É a ferramenta central para atingir os 70% de cobertura de código exigidos. Sem ela, seria impossível validar as complexas estratégias de desconto (como o "Leve 3 Pague 2") de forma automatizada.

Base de Dados Relacional: Garante transações ACID, assegurando que o stock só é descontado se o pagamento for confirmado (FR-040), evitando perdas de inventário.